

HOA Simplified — DataHub

Schema Design & Data-Cleaning Rationale

Tasks 1 & 2 — BizOps / AI Specialist case study

June 2026

1. Overview

This document explains the design of the HOA Simplified DataHub and the decisions made while cleaning three raw exports into it. The goal was a single, normalized source of truth on **Supabase (Postgres)** that is fully reproducible from the source CSVs, with every cleaning decision recorded rather than applied silently.

The pipeline loads **12 associations**, a board roster, and a vendor list, then powers a live one-page profile generator (Task 3). The stack is Python (psycopg2, Jinja2) with OpenAI for the generated narrative. Schema is versioned SQL; the load is idempotent.

2. Task 1 — Schema Design

Five tables model the domain. Three carry the entities, one is the vendor bridge, and one is a cleaning audit trail:

- **associations** — the master HOA list (1 row per association).
- **board_members** — board roster, FK to associations.
- **vendors** — de-duplicated vendor companies.
- **vendor_associations** — many-to-many bridge between vendors and associations.
- **data_quality_flags** — one row per cleaning decision.

2.1 Keys

Each entity uses a **surrogate** id primary key, plus a unique business key where one exists (hoa_code on associations, email on vendors). The surrogate key keeps foreign keys stable even if a business value is later corrected — a typo in hoa_code can be fixed in one place without rewriting every dependent row.

2.2 Data types & constraints

Money and dates are stored as proper types, never as text. Key columns:

Column	Type	Why
monthly_dues	numeric(10,2)	Currency stored as text in source (\$, commas); parsed to exact decimal.
reserve_balance	numeric(14,2)	Same; nullable for N/A / TBD.
fiscal_year_end_month	smallint (1–12)	Source mixed month names and numbers; one queryable representation.
last_reserve_study_date	date	Four source date formats normalized to a real date.
last_reserve_study_precision	text (day/month)	Records whether a source date was month-only (e.g. 04/2022) so a normalized 1st-of-month never implies false precision.
state	char(2) + regex check	Mixed spellings normalized to a 2-letter code.
role	text + CHECK	Constrained to the five real board roles to catch typos.

2.3 The vendor relationship

A vendor can serve many associations and an association uses many vendors, so the relationship is modeled as a **many-to-many bridge** (vendor_associations) with a composite primary key (vendor_id, association_id) that also prevents duplicate links. Crucially, a **prospect** (a vendor that serves no association yet) is simply a vendor with zero rows in the bridge — it needs no special-casing, no nullable foreign key, and no sentinel value.

2.4 The audit table

The data_quality_flags table records **entity, field, raw value, issue, action, and severity** for every value the loader transforms or cannot trust. This turns the case study's instruction to “flag rather than silently fix” into queryable evidence that lives inside the database, not buried in code comments.

2.5 Scope decisions (deliberate non-normalization)

Two places where further normalization was considered and rejected as over-engineering at this scale:

- **Board members are per-association rows, not a shared person entity.** Some people appear on two boards with the same email (e.g. Avery Cole on SR-07 and SR-09). A dedicated person table was rejected for 12 HOAs; the observation is recorded instead.
- **Roles and trades use CHECK constraints, not lookup tables.** At this scale, lookup tables add joins without real benefit. CHECK constraints still enforce the valid set.

3. Task 2 — Data Ingestion & Cleaning

The loader (load.py) parses each CSV with a real CSV reader (quoted commas in names are preserved), normalizes every field, de-duplicates vendors, and writes an audit row for each decision. It truncates and reloads on every run, so the DataHub is reproducible from the CSVs at any time.

3.1 Field normalization

Field	Source mess	Decision
Money (dues, reserve)	\$285.00, 61,200, N/A, TBD	Strip \$ / commas → numeric; placeholders → NULL (flagged).
State	CA / Calif. / California / ca	Normalize to CA.
Fiscal year end	December / 12 / blank	Month number 1–12; blank → NULL.
Reserve study date	2023-04-12, 04/2022, March 2024, N/A	Parse all four; month-only → 1st of month + precision flag.
has_reserve_study	Yes / No / yes	Boolean.
Board term dates	ISO and MM/DD/YYYY	Normalize to date; blank end = currently serving.
Phone	5 formats	Normalize to (NNN) NNN-NNNN.

coi_on_file	Yes / Y / yes / No / blank	Boolean; on merge keep the most-complete value.
serves_hoa_codes	“SR-01; SR-04; SR-10” in one cell	Split into individual bridge rows.

3.2 Vendor de-duplication

The intake has 14 vendor rows but fewer real companies — the same vendor recurs with different name suffixes, phone formats, and casing (e.g. “Bright Path Landscaping” vs “Bright Path Landscaping LLC”). **Email is the de-duplication key**, because it is the one field that stays consistent across a vendor’s rows, whereas a fuzzy name match would be fragile. For each email group the loader: chooses a canonical name (most frequent, shortest as tie-break, which drops the legal suffix), unions the served-HOA codes and service areas, takes the most-complete COI value, and records the merge.

Result: **14 rows → 10 real vendors**, with served-HOA sets preserved exactly. For example, the three Bright Path rows merge into one vendor serving SR-01, SR-04, SR-07, and SR-10.

3.3 Prospects

Vendors that serve no association (blank `serves_hoa_codes`) are loaded as normal vendor rows with zero bridge rows, and flagged as prospects. Four vendors are prospects. They remain first-class records, ready to be linked later.

3.4 Flag, don’t silently fix

Every transformation and every nulled value is recorded — **42 flags** in total (38 informational, 4 warnings). A follow-up audit confirmed that **every NULL in the database traces to a flag**. That audit caught two values that were being nulled silently — a missing reserve-study date and a missing association board email — which are now flagged for parity with the analogous fields. Nothing is changed without a recorded reason.

3.5 What we flagged rather than “corrected”

Some values are ambiguous; rather than guess, the loader keeps the data and raises a flag for a human:

- **\$0 monthly dues (SR-06)** — kept, flagged as “verify” (could be a genuine new/prospect HOA, not an error).
- **N/A and TBD reserves** — nulled and flagged, not coerced to 0.
- **Same person on multiple boards** — recorded as an observation; not merged into one identity.

4. Task 3 — One-Pager (how it ties in)

The generator reads **live from the database** (never the CSVs) and renders a self-contained HTML profile. The “areas to watch” section is written by an LLM, but follows a deliberate principle: **facts in code, language in the LLM**. Every figure and risk flag (reserve-per-unit, missing study, incomplete board, expired terms, insurance gaps) is computed deterministically; the model only writes prose grounded in those computed facts, which prevents hallucinated numbers. Missing data degrades gracefully (e.g. “Reserve study: not on file”).

5. Use of AI Tools (Transparency)

AI was used throughout, as the case study encourages, with human review and correction at each step:

- **Code generation** (schema, cleaning pipeline, generator) was AI-assisted and reviewed line by line. The cleaning rules, the email-based dedup strategy, and the schema’s key choices were directed and corrected by hand.
- **Runtime narrative** uses OpenAI (gpt-4o-mini), constrained to the computed structured facts described in Section 4.
- **Where AI output was overridden:** a misleading data-source label on the report was caught and corrected; an over-claim about output determinism was caught and documented honestly (see below); and two silently-nulled fields were promoted to explicit flags rather than left as the first pass produced them.

On determinism: OpenAI does not guarantee identical output even at `temperature=0` with a fixed seed (best-effort only). The **substance** of every report is fully deterministic because the facts are computed in code; only the prose wording can vary slightly between runs. This is stated plainly rather than papered over.

6. Key Tradeoffs

Decision	Reasoning
Surrogate id + unique business key	Stable FKs; business values can be corrected without cascading rewrites.
CHECK constraints over lookup tables	Enforces valid sets without join overhead at 12-HOA scale.
Board members per-association (no person entity)	Avoids over-engineering; cross-board identity recorded as an observation.
Email as vendor dedup key	The only consistently clean field; safer than fuzzy name matching.
Flag rather than fix ambiguous values	Preserves the client’s data and surfaces decisions for human review.
HTML one-pager	Self-contained, presentable, easy to share; PDF is a one-line add if needed.

Reproduce: create the schema (migrations/001_schema.sql), run load.py, then generate.py. The data_quality_flags table records every cleaning decision. See README.md for the full runbook.